EXPERIMENT NO: 09

Roman Numeral to Integer Conversion

1. Basic Information

Program Name: Experiment9.java

Package: recordprog

Language: Java

2. Objective / Aim

To develop a Java program that accepts a Roman numeral from the user, converts it into its equivalent integer value, and displays the result. The program also handles empty input and invalid Roman numeral characters using exception handling.

3. Description

The program reads a Roman numeral, removes extra spaces, and converts the input to uppercase. It checks whether the input is empty and displays an appropriate message if it is. A separate **Roman** class converts the numeral to an integer by comparing each character's value with the value of the character to its right. If a smaller value appears before a larger value, it is subtracted; otherwise, it is added. Invalid characters are handled by throwing an IllegalArgumentException.

4. System Requirements

Hardware: Standard PC/Laptop; minimum 4 GB RAM; minimum 500 MB free storage.

Software: Windows/Linux/macOS; Java Development Kit (JDK); IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code; Command Prompt/Terminal.

5. Algorithm

1. Start the program.

2. Read a Roman numeral from the user.

3. Remove leading and trailing spaces and convert the input to uppercase.

4. If the input is empty, throw an IllegalArgumentException.

5. Initialize total and previous value to zero.

6. Read the Roman numeral characters from right to left.

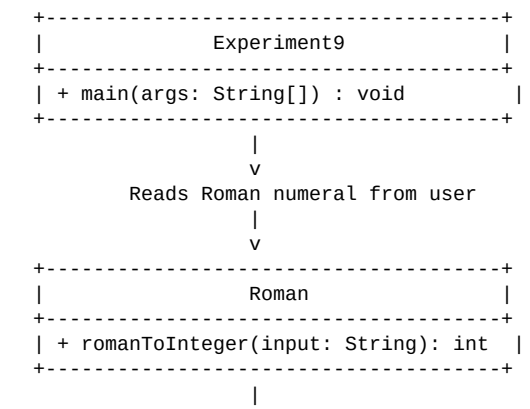
7. If a character is not a valid Roman numeral symbol, throw an IllegalArgumentException.

8. If the current symbol's value is less than the previous value, subtract it from total; otherwise, add it.

9. Display the integer value.

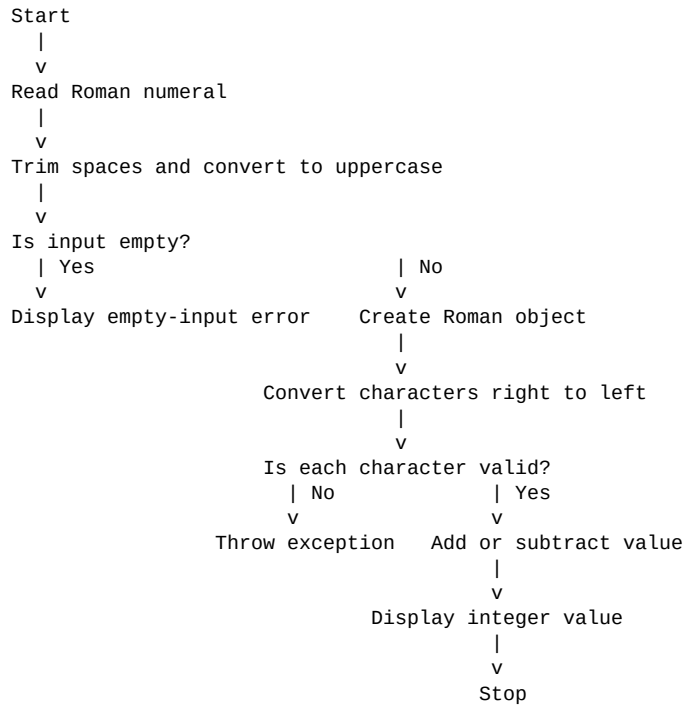
10. Catch and display any IllegalArgumentException message, close the scanner, and stop.

6. UML / Program Structure



```
            v
Converts Roman numeral to integer
```

7. Flow of Execution



8. Source Code

Experiment9.java

```
package recordprog;
import java.util.Scanner;

public class Experiment9 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a Roman numeral: ");
        String input = sc.nextLine().trim().toUpperCase();
        try {
            if (input.isEmpty()) {
                throw new IllegalArgumentException(
                    "Roman numeral cannot be empty."
                );
            }
            Roman roman = new Roman();
            int result = roman.romanToInteger(input);
            System.out.println("Integer value: " + result);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }
        sc.close();
    }
}
```

Roman.java

```
package recordprog;
import java.util.Map;

public class Roman {
    private static final Map<Character, Integer> VALUES = Map.of(
        'I', 1, 'V', 5, 'X', 10, 'L', 50,
        'C', 100, 'D', 500, 'M', 1000
    );

    public int romanToInteger(String input) {
        int total = 0;
        int previous = 0;
        for (int i = input.length() - 1; i >= 0; i--) {
            char ch = input.charAt(i);
            if (!VALUES.containsKey(ch)) {
                throw new IllegalArgumentException(
                    "Invalid Roman numeral character: " + ch
                );
            }
```

```
            int value = VALUES.get(ch);
            if (value < previous) {
                total -= value;
            } else {
                total += value;
            }
            previous = value;
        }
        return total;
    }
}
```

9. Compilation and Execution

```
javac -d . Experiment9.java Roman.java
java recordprog.Experiment9
```

10. Sample Output

Example 1:

```
Enter a Roman numeral: XIV
Integer value: 14
```

Example 2:

```
Enter a Roman numeral: MCMXCIV
Integer value: 1994
```

Example 3:

```
Enter a Roman numeral:
Roman numeral cannot be empty.
```

Example 4:

```
Enter a Roman numeral: AB
Invalid Roman numeral character: B
```

11. Result

Thus, the Java program to convert a Roman numeral into its integer equivalent and handle empty input or invalid characters using exception handling was written and executed successfully.